Supermarket Enterprise Management System

COSC 3380 Database Systems Design • Team 22

Executive Summary & System Architecture

This report presents the preliminary database system design for Team 22's Supermarket Chain Enterprise application. The system models a distributed multi-store supermarket enterprise operating across multiple state jurisdictions. The architecture incorporates an OLTP customer checkout simulator that enforces atomic multi-item basket processing, point-of-sale inventory depletion, dynamic state sales tax calculation, and two-phase electronic bank account settlement. The web-based operational dashboard provides real-time access to database management procedures, schema DDL execution, table browsing, and managerial analytics.

Team Member Work Distribution

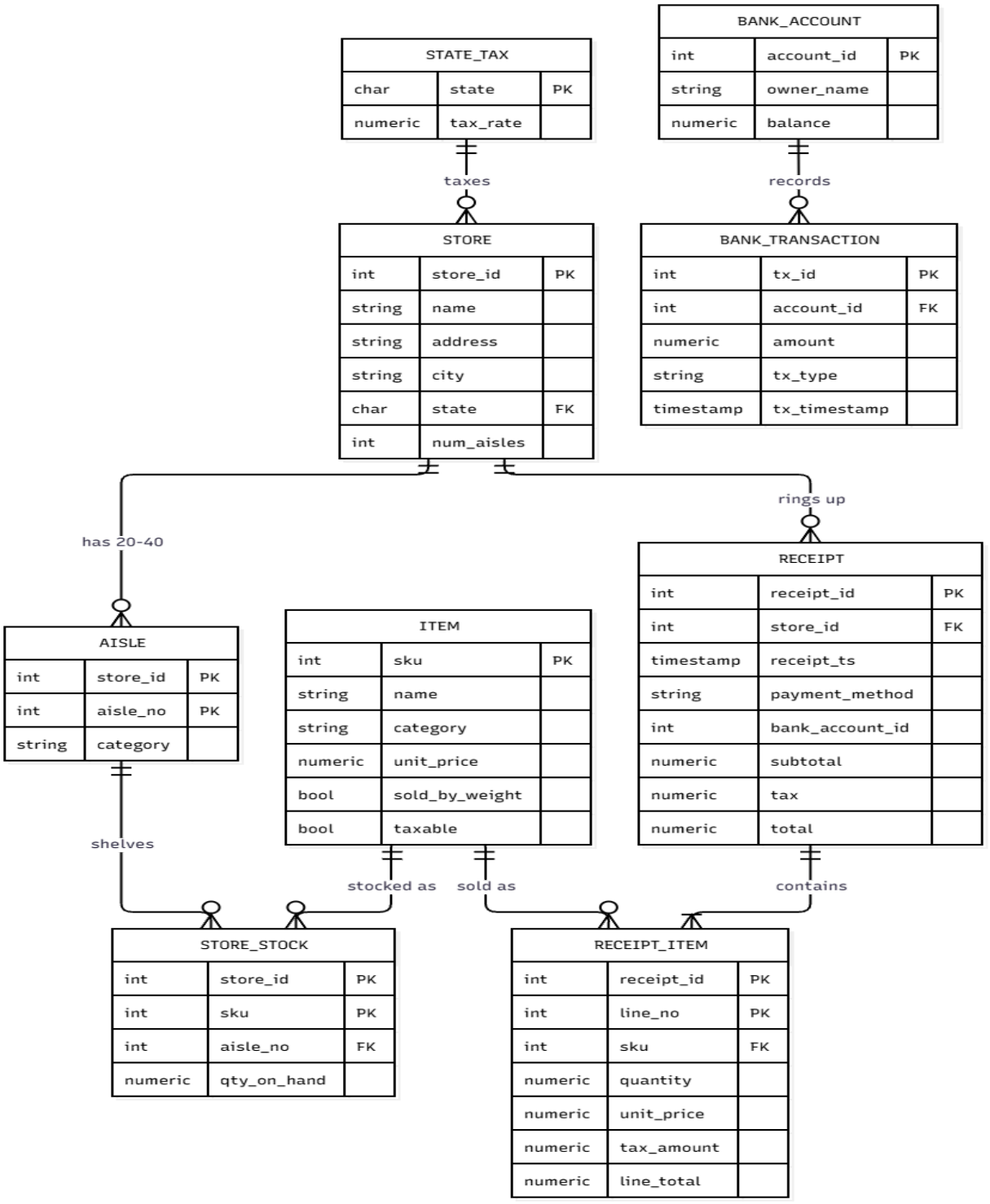
Team Member	Assigned Role	Core Contributions
Trey Langley	Full-Stack & Server Deployment	Local environment setup, server-side Node.js routing, course server SSH/SCP deployment, SQL trace generation (`transaction.sql`, `query.sql`).
Miguel A. Duran	Database Architect & DDL Lead	Relational schema design (`schema.sql`), BCNF normalization proofs, seed dataset generation (`init.sql`), GitHub repository management.
Youssef Selim	Front-End & Domain Analyst	Web GUI front-desk interface (`home.html`), empirical receipt data collection (5 chains), field mapping synthesis, and report documentation.

Relational Schema Architecture (9 Normalized Relations)

Relation Name	Primary Key (PK)	Foreign Keys (FK)	Domain & Storage Scope
state_tax	state (CHAR(2))	None	State-level sales tax rate definitions (TX, CA, NY, FL).
store	store_id (INT)	state → state_tax	Physical branch locations (10 stores) with ≥ 20 aisles.
aisle	(store_id, aisle_no)	store_id → store	Composite key mapping 20–32 numbered store aisles.
item	sku (INT)	None	Product catalog, category, unit price, weight/tax flags.
store_stock	(store_id, sku)	sku , (store_id, aisle_no)	Live shelf inventory quantities per store branch and aisle.
receipt	receipt_id (SERIAL)	store_id → store	Checkout header: timestamp, tender, subtotal, tax, total.
receipt_item	(receipt_id, line_no)	receipt_id , sku → item	Line-item basket detail with historical price Snapshot.
bank.account	account_id (INT)	None	Electronic banking accounts and live balances for customers and corporate treasury.
bank.bank_transaction	tx_id (SERIAL)	account_id → bank.account	Immutable transaction ledger recording debits/credits (`WITHDRAW`/`DEPOSIT`), amounts, and timestamps.

Preliminary Entity-Relationship (ER) Model in UML Notation

The conceptual model below specifies the complete relational database architecture for Team 22's Supermarket Chain enterprise. Designed in standard UML class diagram notation using draw.io (Supermarket_er.drawio), the model defines nine normalized entities across physical store topology, localized tax rules, catalog items, shelf inventory, checkout receipts, and multi-party financial accounts. Every primary key (PK), foreign key (FK), data type, and cardinality constraint is explicitly specified.



Structural Cardinalities & Foreign Key Integrity

Relationship Path	Cardinality	Enforcement Rule	Operational Semantic Justification
state_tax → store	1 : N	REFERENCES state_tax(state)	Every store belongs to exactly one state; states govern local sales tax.
store → aisle	1 : N (20–40)	ON DELETE CASCADE	A store maintains 20–40 distinct physical aisles; removing a store purges its aisles.
item & aisle → store_stock	N : M	FOREIGN KEY (store_id, aisle_no)	Items are shelved in specific store aisles with independent on-hand inventory levels.
store → receipt	1 : N	REFERENCES store(store_id)	A store issues many receipts; receipts uniquely identify their origination branch.
receipt → receipt_item	1 : N (10–20)	ON DELETE CASCADE	Each checkout receipt contains 10 to 20 individual line items.
item → receipt_item	1 : N	REFERENCES item(sku)	Items are scanned into receipt line items; catalog deletion is restricted.
bank.account → bank_tx	1 : N	REFERENCES bank.account(id)	Customer accounts maintain an immutable ledger of electronic debit transactions.

Domain Constraints & Schema Invariants

Enforced Business Logic Invariants:

Aisle Scaling Invariant: CHECK (num_aisles >= 20) guarantees each supermarket branch qualifies as an enterprise facility.
Inventory Depletion Invariant: CHECK (qty_on_hand >= 0) prevents negative stock balances during simultaneous checkout transactions.
Solvency Invariant: CHECK (balance >= 0) on bank.account prevents overdrafting; purchases abort if funds are insufficient.
Unit Scale Distinction: sold_by_weight = TRUE allows fractional quantities (e.g., 2.14 lbs); packaged items enforce whole counts.

Relational Performance & Secondary Indexes

B-Tree Secondary Indexing Architecture:

CREATE INDEX idx_receipt_store ON receipt(store_id); — Speeds up branch revenue aggregations and chronological receipt filtering.
CREATE INDEX idx_stock_store_sku ON store_stock(store_id, sku); — Accelerates inventory lookups and point-of-sale decrement operations during high-frequency customer checkouts.
CREATE INDEX idx_receipt_item_sku ON receipt_item(sku); — Optimizes enterprise reporting queries when evaluating top-selling items across historical transactions.

Empirical Analysis of Five Supermarket Receipts

To ensure the relational model accurately captures commercial supermarket operations, we collected and analyzed five physical and digital receipts from major national and regional chains: H-E-B Grocery Company, The Kroger Company, Walmart Supercenter, Trader Joe's, and Whole Foods Market. Every field on these receipts maps directly into our schema entities: store identification, aisle classification, SKU lookup, scale weights, tax status flags, subtotal aggregations, and electronic payment ledger references.

Receipt Data Token to Relational Schema Mapping

Receipt Token	Observed Commercial Examples	Relational Attribute	Database Semantics & Type Logic
Store Header	H-E-B #128, 5106 Bissonnet St, Bellaire TX Kroger #034, 1938 W Gray, Houston TX	store(store_id, address, city, state)	Maps branch number, street, city, and state code; anchors geographic tax rate lookup.
Product / SKU	H-E-B: "ORG BANANAS 4011" Walmart: "007874235182 GV MILK 1G"	item(sku, name, category)	Standardized PLU/UPC barcode numbers stored as integer primary keys in item master.
Weighed Goods	Kroger: "Honeycrisp Apples 1.84 lb @ 2.49/lb" H-E-B: "Ribeye Steak 1.42 lb @ 14.99/lb"	receipt_item.quantity (NUMERIC(8,2)) item.sold_by_weight = TRUE	Decimal quantity representing certified scale weight multiplied by unit price per pound.
Packaged Unit	Trader Joe's: "Sourdough Bread 1 @ 4.99" Whole Foods: "365 Eggs 12ct 1 @ 4.19"	receipt_item.quantity (NUMERIC(8,2)) item.sold_by_weight = FALSE	Discrete integer count representing pre-packaged individual consumer units.
Tax Code Flag	H-E-B: "F" (Food/Exempt), "T" (Taxable) Walmart: "N" (Non-tax), "T" (Taxable 8.25%)	item.taxable (BOOLEAN) receipt_item.tax_amount	Under Texas Tax Code § 151.314, staple groceries are zero-rated; merchandise is taxable.
Financial Totals	"SUBTOTAL $48.20 • TAX $1.65 • TOTAL $49.85"	receipt(subtotal, tax, total)	Precalculated transaction summaries derived from line totals to enable fast OLTP queries.
Tender Method	"VISA DEBIT AUTH 883921 AID A0000000031010" Cash tendered: "$50.00 CHANGE $0.15"	receipt.payment_method receipt.bank_account_id	Enum-constrained payment mechanism; card purchases link to bank debit accounts.
Aisle Mapping	Kroger Mobile App: "Aisle 14 - Pantry" H-E-B App: "Aisle 7 - Dairy & Eggs"	store_stock.aisle_no aisle.category	Connects inventoried stock to physical supermarket store geometry (20–40 aisles).

Critical Commercial Observations on Taxation & Item Units

1. Grocery Tax Jurisdiction Rules: In Texas, staple food items (unprepared produce, meat, dairy, flour) are exempt from sales tax, whereas carbonated beverages, snacks, and household cleaning chemicals incur the 8.25% state and local surcharge. Our schema implements this using a boolean item.taxable flag combined with the relational join to state_tax.

2. Variable Weight vs. Discrete Pricing: Supermarkets compute lines differently depending on packaging. A weighed item computes line_total = ROUND(weight × unit_price, 2), requiring high-precision NUMERIC(8,2) representation rather than integer units to avoid commercial truncation errors.

Relational Normalization & BCNF Proof

A relational schema R is in Boyce-Codd Normal Form (BCNF) if, for every non-trivial functional dependency X → Y, X is a superkey of R. We verify BCNF compliance across our core relations:

store: FD is store_id → name, address, city, state, num_aisles. Determinant store_id is the primary key (superkey). Hence, BCNF holds.

aisle: FD is (store_id, aisle_no) → category. Determinant is the candidate key. Hence, BCNF holds. item: FD is sku → name, category, unit_price, sold_by_weight, taxable. Determinant sku is a superkey. BCNF holds.

store_stock: FD is (store_id, sku) → aisle_no, qty_on_hand. Determinant is the composite primary key. BCNF holds.

bank.account: FD is account_id → owner_name, balance. Determinant is a superkey. BCNF holds.

Justification of Intentional 3NF / BCNF Exceptions

Documented Denormalization Justifications:
1. Historical Price Snapshot (receipt_item.unit_price): Strictly speaking, sku → unit_price exists in item. However, product catalog prices fluctuate over time. If receipt_item referenced item.unit_price via join, all historical receipts would retroactively mutate whenever store prices change. Preserving point-in-time unit prices and line totals directly on the receipt line is mandatory for commercial auditability and legal accounting standards.

2. Derived Financial Summaries (receipt.subtotal, tax, total): While derived from ∑(receipt_item.line_total), persisting transaction totals prevents costly repeated aggregations during high throughput daily sales reporting, cashier reconciliation, and managerial queries.

Front Desk ACID Checkout Simulation & Analytical Queries

The customer checkout procedure executes as an atomic ACID transaction within server.js. The transaction performs five coordinated operations inside a single database connection:

```
BEGIN TRANSACTION; -- 1. Insert receipt header and retrieve generated receipt_id INSERT INTO receipt (store_id, payment_method, bank_account_id, subtotal, tax, total) VALUES (...); -- 2. Verify and decrement store shelf stock (enforces qty_on_hand >= 0 check) UPDATE store_stock SET qty_on_hand = qty_on_hand - :qty WHERE store_id = :sid AND sku = :sku; -- 3. Append scanned items into receipt_item ledger with historical price snapshot INSERT INTO receipt_item (receipt_id, line_no, sku, quantity, unit_price, tax_amount, line_total) VALUES (...); -- 4. Execute bank transfer: debit customer account and credit corporate account UPDATE bank.account SET balance = balance - :total WHERE account_id = :cust_acc; UPDATE bank.account SET balance = balance + :total WHERE account_id = 1; -- 5. Commit atomic transaction (or ROLLBACK if balance or stock is insufficient) COMMIT;
```

The analytical queries in query.sql evaluate multi-table aggregations, calculating state-by-state gross revenue, identifying the top ten volume items using ORDER BY SUM(quantity) DESC, and triggering replenishment alerts when qty_on_hand < 50.00.